Orbit: Approximate Prefix-Aware Routing for Multi-Cluster LLM Serving

Final Project Report

Abstract

Large language model (LLM) serving systems increasingly rely on key-value (KV) cache reuse to reduce prefill work and time-to-first-token (TTFT). Existing routing mechanisms either operate inside a single serving engine, route semantically across models, or assume relatively direct knowledge of cache state. This report presents Orbit, a cluster-level router that uses compact hierarchical prefix summaries to approximate where reusable prompt prefixes are likely cached. Each serving cluster exports Bloom-filter summaries and hot-prefix sketches; routers combine these approximate locality signals with visible load, summary staleness, uncertainty, and network latency. Orbitis evaluated on the most recent benchmark run in `mixed-external-renamed-baselines-large-v7-20260420`, using a live `llama.cpp` backend, a sparse four-router six-cluster topology, and mixed external ShareGPT-style chat, RAG, and agent/tool traffic. Across five seeds, Orbitreduces median TTFT from 3.17s under least-loaded routing to 0.56s, reduces median end-to-end latency from 5.01s to 2.51s, and increases mean reusable prefix length from 77.6 to 100.1 tokens. Compared with random and round-robin routing, Orbitsimultaneously improves latency and cache reuse. These results support the thesis that approximate, hierarchical prefix summaries are sufficient to expose useful KV-cache locality at the inter-cluster routing layer without requiring exact global KV state.

1 Introduction

LLM inference has a two-phase structure. The prefill phase processes the input prompt and builds KV cache state, while the decode phase emits continuation tokens. For repeated prompts, multi-turn conversations, RAG templates, and agent workflows, prefill work is often redundant. Modern serving engines exploit this observation through prefix caching, KV cache managers, and memory-efficient attention kernels. However, many deployments do not consist of one monolithic engine. A realistic service may contain multiple clusters, multiple router processes, sparse reachability between routers and clusters, stale control-plane state, and uneven queueing pressure. In that setting, a request can be sent to a lightly loaded cluster that has no useful prefix state, or to a cluster with good prefix locality but unacceptable queue delay.

Orbitaddresses this routing problem at the cluster boundary. The system is not a replacement for vLLM, `llama.cpp`, TensorRT-LLM, or other single-node engines. Instead, it sits above serving clusters and decides which reachable cluster should receive each request. The design is motivated by a simple constraint: exact global KV-cache state is too expensive and too dynamic to replicate to every router. A router therefore needs a compact approximation that is good enough to distinguish likely cache hits from likely misses.

The core contribution of Orbitis a routing policy that combines approximate prefix locality with a TTFT-oriented cost model. Clusters maintain exact local prefix state, but export only summaries. A router probes those summaries at multiple prefix depths and estimates how many input tokens are likely reusable. It then predicts TTFT and total latency using the estimated remaining prefill tokens, visible queue depth, summary age, route uncertainty, and router-to-cluster network cost. The final decision is conservative: Orbitcomputes a least-loaded fallback and only chooses the prefix-aware route when the summary signal is fresh and sufficiently advantageous.

This report frames the current implementation as a research artifact. It describes related work, implementation details, evaluation methodology, latest results, limitations, and future directions. The report references the newest benchmark artifacts currently present in the repository: `mixed-external-renamed-baselines-large-v7-20260420`. That run was generated on 2026-04-20 with a live `llama.cpp` backend and five seeds. It uses the default configuration now encoded in the project.

1.1 Contributions

This work makes three contributions. First, it designs an inter-cluster routing abstraction for prefix-cached LLM serving in which clusters export compact summaries rather than exact KV-cache contents. This is the main separation from engine-internal prefix routers: Orbitassumes cache state is local, fast-changing, and only approximately visible to remote routers. Second, it implements a routing policy that combines estimated prefix reuse, queue depth, network cost, summary age, and uncertainty into a TTFT-oriented decision rule with least-loaded fallback. The fallback is part of the design, not an implementation detail, because approximate summaries can be wrong. Third, it evaluates the router on a mixed external workload built from ShareGPT-style chat, RAG, and agent/tool prompts, using a live `llama.cpp` backend and a sparse multi-router topology. The evaluation compares Orbit

only against baselines that match the implemented deployment boundary: least-loaded, random, and round-robin inter-cluster routing.

2 Related Work

vLLM and PagedAttention. vLLM introduced PagedAttention, a virtual-memory-inspired KV-cache management technique that reduces fragmentation and supports flexible sharing of KV cache within and across requests [1]. vLLM's contribution is primarily within the serving engine: it manages KV blocks efficiently so high-throughput batching is possible. Orbitassumes such engine-level cache mechanisms exist, but asks a different question: when there are multiple clusters, which cluster should receive a request so that its engine-local KV cache is most useful?

vLLM prefix-aware and KV-cache-aware routing. The vLLM Production Stack includes prefix-aware routing and KV-cache-aware routing tutorials [2, 3]. Prefix-aware routing tries to send subsequent requests with the same prefix to the same instance. KV-cache-aware routing extends this idea by routing to the instance with the best cache hit rate rather than blindly pinning by prefix. These systems are the closest operational analogues to Orbit. The difference is scope and information model. vLLM's production-stack router targets instances in a Kubernetes deployment and integrates with LMCache/vLLM state. Orbittargets an inter-cluster setting with sparse router-cluster reachability and intentionally does not require exact global KV state. Its summaries are approximate and hierarchical, so they are cheaper to gossip and tolerate staleness.

vLLM Semantic Router. vLLM Semantic Router is an intelligent routing layer for Mixture-of-Models deployments. It extracts request signals such as keywords, embeddings, domain classifications, safety signals, and user preferences, then selects an appropriate model or plugin path [4, 5]. This is complementary to Orbit. Semantic routing chooses *what* model or policy should answer a request; Orbitchooses *where* within a cluster pool a request should execute to exploit prefix reuse while respecting load and latency.

Helix and heterogeneous serving optimization. Helix formulates distributed LLM serving over heterogeneous GPUs and network links as a max-flow optimization problem and uses MILP planning to jointly optimize model placement and request scheduling [6]. Helix operates at a deeper resource-planning layer than Orbit: it reasons about GPU heterogeneity, network capacities, and per-request pipelines. Orbitinstead assumes each cluster is already a serving unit and focuses on dynamic request routing with prefix locality. A practical deployment could combine the two: Helix-like planning could provision and place serving pipelines, while Orbitroutes repeated-prompt traffic among reachable clusters.

Disaggregated prefill, decode, and KV cache. DistServe and Splitwise split prefill and decode phases across resources to reduce phase interference and improve goodput, throughput, cost, or power [7, 8]. Mooncake pushes further toward a KV-cache-centric disaggregated architecture, separating prefill and decode clusters and using CPU, DRAM, SSD, and NIC resources as part of a global KV-cache substrate [9]. NVIDIA Dynamo documents production mechanisms for disaggregated serving, KV transfer, and KV cache routing [10]. These systems move or share KV cache across components. Orbitmakes a different tradeoff: it avoids transferring exact cache metadata globally and instead routes whole requests using compact approximate summaries. This makes Orbitlightweight, but less powerful than a system that can directly fetch or migrate KV blocks.

Positioning. Table 1 summarizes the distinction. Orbitis not the first system to exploit prefix reuse, and it does not compete with engine-level cache managers. Its contribution is an inter-cluster, approximate-information routing layer that uses hierarchical summaries to balance locality and load under stale control-plane state.

3 Implementation

3.1 System Model

Orbitmodels a deployment with R routers and C serving clusters. In the final benchmark, $R = 4$ and $C = 6$. The topology is sparse: each router can reach only three clusters. Each cluster runs the same model, but network costs differ by router-cluster pair. This reflects a multi-cluster deployment where locality, reachability, and load interact.

Each request x has a router id, input prefix tokens, prompt text, continuation token budget, traffic class, and optional session/source identifiers. A router observes a set of reachable clusters and selects one policy decision. The evaluated policies are:

- **Orbit** (`summary` in artifacts): prefix-aware routing using approximate summaries and a load-aware cost model.
- **Least Loaded** (`load_only`): routes by visible queue/load and latency cost, ignoring prefix locality.
- **Random**: selects a reachable cluster uniformly.
- **Round Robin**: cycles through reachable clusters without using load or prefix state.

3.2 Design Goals

The implementation is organized around three design goals. The first is *engine independence*. Orbit should not depend on vLLM block tables, `llama.cpp` cache internals, or TensorRT-LLM executor structures. It only needs token ids, stable prefix hashes, request completion events, and coarse

Table 1: Positioning relative to nearby systems. Orbit targets the *inter-cluster, approximate-information* routing layer.

System	Main objective	Routing information	Orbit comparison
vLLM / PagedAttention	Efficient KV memory management inside one engine	Engine-local KV block state	Complementary: Orbit dispatches requests before engine execution.
vLLM prefix-aware router	Preserve prefix locality across serving instances	Prefix or LMCache state for reachable pods	Same locality goal, but Orbit uses approximate cross-cluster summaries instead of exact pod state.
vLLM KV-cache-aware router	Maximize instance-level cache hit rate	Cache-hit telemetry per instance	Orbit avoids requiring exact cache telemetry from every reachable cluster.
vLLM Semantic Router	Model, plugin, and policy selection	Embeddings, safety signals, user preferences	Orthogonal: Orbit selects the serving cluster after model choice.
Helix	Heterogeneous placement and scheduling	GPU/network capacity graph and optimization plan	Orbit performs online dispatch, not capacity planning.
DistServe / Splitwise	Prefill/decode phase disaggregation	Phase-specific resource and queue state	Orthogonal: Orbit routes complete requests.
Mooncake / Dynamo	KV-centric disaggregated serving	Distributed KV index and transfer fabric	Orbit avoids exact global KV state and can sit above these systems.

cluster load. The second is *low control-plane cost*. A router should be able to rank clusters without receiving a full prefix trie or a complete KV-block index from every cluster. The third is *safe degradation*. When summaries are stale, missing, or weak, the router should behave like a conventional load-aware balancer rather than making an aggressive locality decision.

These goals explain two choices that may appear conservative. Orbit routes complete requests rather than splitting prefill and decode, because the experiment isolates the routing layer above a normal serving backend. Orbit also treats network cost as router-cluster latency rather than as a detailed bandwidth model, because the current artifact emulates a sparse cluster topology on one host. A production implementation could replace these terms with measured RTT, available bandwidth, or backend-reported queueing delay without changing the summary abstraction.

3.3 End-to-End Request Flow

The runtime path is intentionally simple. First, the benchmark generates a mixed workload and, for the live backend, tokenizes each prompt using the same `llama.cpp` tokenizer that will serve the request. Second, each cluster periodically publishes a summary to its home router. Third, routers gossip summaries to other routers. Fourth, when a request arrives, the responsible router filters the candidate set to clusters reachable from that router. Fifth, the policy computes a decision and sends the request to the selected cluster. Sixth, after prompt evaluation completes, the cluster inserts the request prefix into its exact local cache state so later requests can reuse it.

This flow separates three concerns that are often coupled in prototype evaluations. The backend controls actual tokenization and completion latency. The cluster controls exact local cache state. The router controls only the decision based on approximate summaries and visible load. This separation is useful because it makes the routing contribution measurable: all policies run the same backend, the same workload, the same sparse topology, and the same cache implementation.

3.4 Why Approximate Rather Than Exact State?

Exact global cache state is attractive because it could identify the best prefix match directly. It is also expensive. KV caches change whenever prompt evaluation completes, and each cluster may cache many prefixes at many lengths. Replicating that state to every router would add bandwidth, memory, and synchronization overhead. Worse, exact state would still become stale between updates. Orbittherefore treats staleness as unavoidable and asks for the smallest useful signal. A Bloom-filter summary can say that a prefix of length d is likely present; a hotset can make common short prefixes exact; a queue-depth field can expose load. Together, these fields are sufficient for a ranking decision even though they are not sufficient to reconstruct cache contents.

The design also avoids tying the router to one serving engine's internal KV-cache representation. vLLM, `llama.cpp`, TensorRT-LLM, and SGLang differ in block layout, eviction, and cache APIs. Prefix summaries only require stable token ids and prefix hashes. That makes the router portable, provided each backend can expose or approximate cache-ready events.

3.5 Cluster Summaries

Each cluster maintains exact local cache state in a prefix trie. The trie is ground truth for evaluation: it can answer the true longest reusable prefix for a request. This state is not exported directly. Instead, at summary intervals, the cluster publishes:

- Bloom filters over prefix hashes at configured depths, e.g., 4, 6, 8, ..., 512 tokens.
- Hotsets of frequent short-prefix hashes, which reduce uncertainty for common prefixes.

- Visible queue depth and summary metadata such as creation time and version.

The summary is compact and approximate. Bloom filters can produce false positives, and summaries can be stale. Both facts are first-class in the routing cost model. Summary memory is recorded in the benchmark artifacts; the latest run reports roughly 991–993 KB of summary memory across routers depending on policy.

3.6 Prefix Reuse Estimation

When a request arrives, a router hashes the request prefix at the summary depths and probes each reachable cluster summary from shallow to deep. If a depth is found in the hotset or Bloom filter, the router treats that depth as a possible match. The first missing level bounds the estimate. If the deepest matched level is d and the next missing level is d', the raw estimate is interpolated between them, capped by request length. A calibrated reuse model can then transform the raw estimate:

$$\hat{k} = \min\left(n, \max\left(0, \alpha_0 + \alpha_1 k_{\text{raw}} \alpha_2 m + \alpha_3 h\right)\right), \quad (1)$$

where n is input length, k_{raw} is the raw prefix estimate, m is the number of matched summary levels, and h is the number of hotset matched levels. In the latest run, validation selected the base configuration rather than the calibrated configuration, so the reported final policy uses the default reuse transformation. This is important: the system includes auto-tuning, but the validation guardrail prevented a tuned model that improved prediction error from harming tail latency.

3.7 Latency and Route Cost Model

For a candidate cluster j, the router estimates remaining prefill tokens as

$$p_j = \max(0, n - \hat{k}_j). \quad (2)$$

It then predicts TTFT:

$$\widehat{\text{TTFT}}_j = r_j + b_0 + b_q q_j + b_p p_j + b_a a_j + b_u u_j + b_m M_j, \quad (3)$$

where r_j is network cost, q_j is visible queue depth, a_j is summary age, u_j is uncertainty gap to the next summary depth, and M_j indicates a missing summary. Total latency is

$$\widehat{L}_j = \widehat{\text{TTFT}}_j + b_d c, \quad (4)$$

where c is the continuation-token budget. Because routing should emphasize time-to-first-token, Orbitranks candidates using

$$\text{RouteCost}_j = \widehat{\text{TTFT}}_j + w(\widehat{L}_j - \widehat{\text{TTFT}}_j). \quad (5)$$

The default w is less than one, so decode cost matters but does not dominate TTFT.

3.8 Fallback and Guardrails

Orbitcomputes a least-loaded route before considering prefix-aware candidates. It falls back to the least-loaded decision when no fresh summary exists, when the best estimated overlap is below a threshold, or when the best prefix-aware candidate does not beat the fallback by a margin. This design avoids overreacting to weak Bloom-filter evidence.

The benchmark also implements warm-up auto-tuning and held-out validation. In the latest run, calibration used 120 warm-up records across five seeds and reduced latency prediction MAE from 770.9 to 0.92. However, the calibrated configuration regressed validation p95 latency and TTFT, so the guardrail rejected it and selected the base router configuration for final evaluation. This behavior is desirable for a research artifact: calibration is allowed to improve prediction accuracy, but not at the cost of tail-latency regressions.

3.9 Backend and Workload

The final evaluation uses a live `llama.cpp` backend with `qwen2.5-3b-instruct-q4_k_m.gguf`. Requests are tokenized by the backend before routing so summary matching uses serving-engine token boundaries. The workload mixes three external traffic classes:

- ShareGPT-style chat from `sharegpt_x_chat.json`.
- RAG traffic from `ragbench_hotpotqa.json`.
- Agent/tool traffic from the ToolBench G1/G2/G3 query data.

The configured weights are 43.75% chat, 31.25% RAG, 25% agent, and 0% standalone bursty traffic. Follow-up bursts can still occur within traffic classes, but there is no separate burst-only dataset in the final run.

3.10 Artifact Layout

Each benchmark run writes a manifest, per-seed workloads, warm-up and validation splits, per-policy request traces, aggregate summaries, calibration output, validation selection output, and PNG figures. The most recent run contains both CSV and JSON traces for every measured policy and seed. This is important for auditability: the aggregate tables in this report can be recomputed from per-request records, and the plots are generated from those same records when available.

The manifest currently records most experimental parameters: backend, topology, seeds, request counts, cache capacity, calibration policy, fault-injection settings, dataset paths, and policy list. One remaining reproducibility gap is that traffic-mix weights are not written into the manifest. The measured counts show that the final run matches the intended mix, but future runs should record the weights explicitly.

4 Evaluation

4.1 Experimental Setup

The newest benchmark directory is `mixed-external-renamed-baselines-large-v7-20260420`. Its manifest records a live `llama.cpp` backend, four routers, six clusters, sparse-overlap topology, three reachable clusters per router, five seeds (7, 11, 17, 23, 29), 144 total requests per seed, 24 warm-up requests, 24 validation requests, and 96 measured requests. The policies are `summary`, `load_only`, `random`, and `round_robin`. The cache token capacity is 4096. Fault injection is disabled in the final run.

The evaluation asks three questions:

1. Does approximate prefix-aware routing reduce TTFT and latency relative to standard baselines?
2. Does it improve prefix reuse at the same time, rather than simply shifting load?
3. Does it remain effective across mixed chat, RAG, and agent traffic?

4.2 Reproducibility Checks

The latest run matches the repository defaults for the recorded manifest fields. The run uses five seeds and 96 measured requests per seed after warm-up and validation, for 480 measured requests per policy. Dataset paths point to the local external datasets under `results/external-datasets-20260418`. The aggregate PNGs used in this report are in the top-level `plots/` directory of the run. Per-seed plots are also present, which allows checking whether a result is driven by one seed or appears consistently.

The validation record is also preserved. It shows that a calibrated latency model was trained from warm-up traces, but the final selected configuration was `base`. This means the final results should be interpreted as the conservative Orbit router, not as a hand-tuned policy chosen after seeing the test split.

4.3 Metrics

The primary metric is TTFT because prefix reuse mostly reduces prefill work, and prefill work determines how quickly a user sees the first generated token. End-to-end latency is reported as a secondary metric because decode time still matters for user-visible completion time. Prefix reuse is measured as reusable input tokens found in the selected cluster's exact local trie, not merely as the router's estimate. This distinction matters: the router makes decisions using summaries, while the evaluator scores reuse using ground truth local cache state. Load standard deviation is included to show whether a policy achieves low latency by overloading a subset of clusters or by distributing traffic evenly.

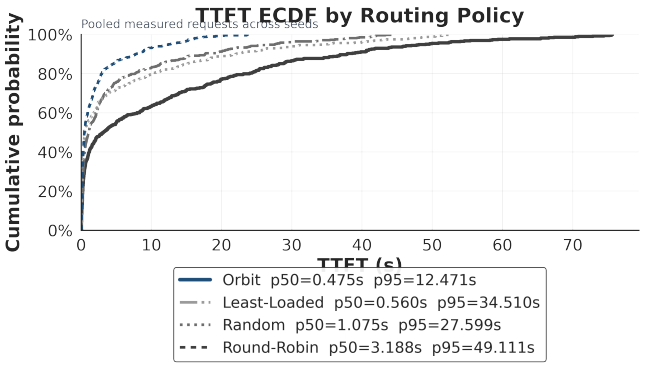

Figure 1: TTFT ECDF for the latest aggregate run. Orbit shifts most of the TTFT distribution left relative to all baselines.

The reported p50 and p95 values are computed per seed and then averaged across seeds. This keeps the five random seeds visible in the aggregate rather than collapsing all requests into one pooled distribution. The ECDF figures are generated from the measured request traces and are useful for checking whether a p50 improvement also shifts the broader latency distribution.

4.4 Aggregate Results

Table 2 reports mean metrics across five seeds. Orbitis labeled `summary` in the artifacts and Orbit in the table. It is best on median TTFT, median latency, p95 TTFT, p95 latency, and mean reusable prefix length. Compared with least-loaded routing, Orbit reduces median TTFT by 82.3%, reduces median end-to-end latency by 49.9%, reduces p95 latency by 52.3%, and increases reusable prefix length by 28.9%. Compared with round robin, Orbit reduces median latency by 78.9% and increases reusable prefix length by 65.2%.

The load standard deviation column highlights an important result. Round robin balances request counts most evenly, but performs worst on latency and prefix reuse. This shows that even load distribution is not sufficient in a prefix-cached LLM serving setting. Least loaded uses queue information but still ignores cache locality. Orbit trades some load balance for substantially better cache locality and lower latency.

4.5 Traffic-Class Breakdown

Table 3 shows Orbit's per-class behavior. Agent traffic has the highest reuse fraction because tool/system preambles and structured prompts repeat strongly. RAG traffic has lower reusable prefix length because retrieved contexts vary by example. ShareGPT-style chat has longer prompts and therefore higher median latency, but still benefits from repeated conversation structure.

4.6 Interpretation

The key finding is not merely that Orbit improves latency. It improves latency while also increasing reusable prefix length.

Table 2: Latest aggregate results from `mixed-external-renamed-baselines-large-v7-20260420`. Values are means across five seeds. Lower is better for TTFT and latency; higher is better for reusable prefix.

Policy	TTFT p50 (s)	TTFT p95 (s)	Latency p50 (s)	Latency p95 (s)	Reusable prefix	Load stddev
Orbit	**0.561**	**10.256**	**2.509**	**11.892**	**100.08**	8.16
Least Loaded	3.175	23.096	5.011	24.952	77.62	9.92
Random	3.349	14.893	5.618	16.683	75.22	4.32
Round Robin	8.832	32.934	11.868	35.473	60.57	**2.76**

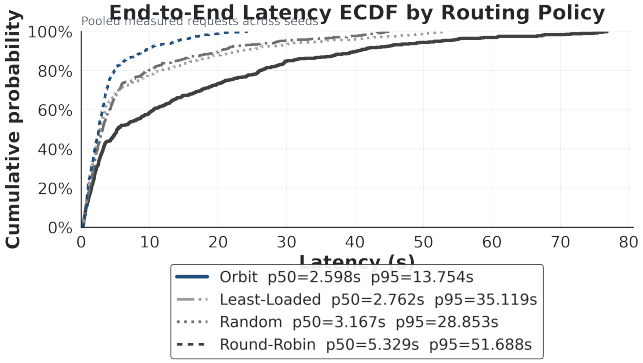

Figure 2: End-to-end latency ECDF. Prefix-aware routing improves the distribution beyond the median, although the longest tails remain workload-dependent.

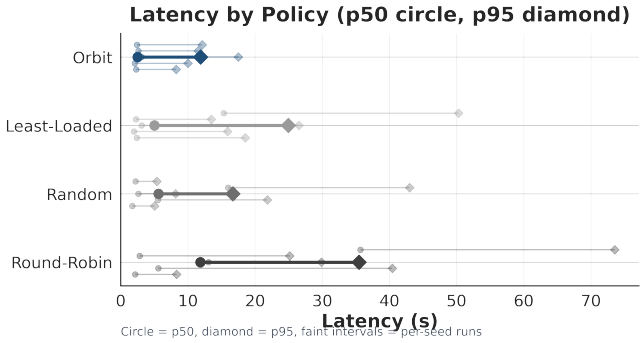

Figure 3: Median and p95 latency by policy. Orbit improves both central tendency and tail latency in the latest benchmark.

That matters because a router could reduce latency by ignoring reuse and chasing the shortest queue, or improve reuse by always pinning sessions and overloading a cluster. Orbit's advantage comes from combining both signals. It chooses prefix-local clusters when summaries indicate meaningful reuse, but rejects prefix-aware candidates when summaries are stale or weak.

The results also clarify why the baselines behave as they do. Random routing occasionally benefits from accidental cache hits and low queue pressure, but it has no systematic locality. Round robin avoids stale load estimates by construction, but it is prefix-blind and performs poorly in the sparse topology. Least loaded reacts to visible queue depth, but in this workload queue snapshots alone are not enough to identify where prefill can be avoided.

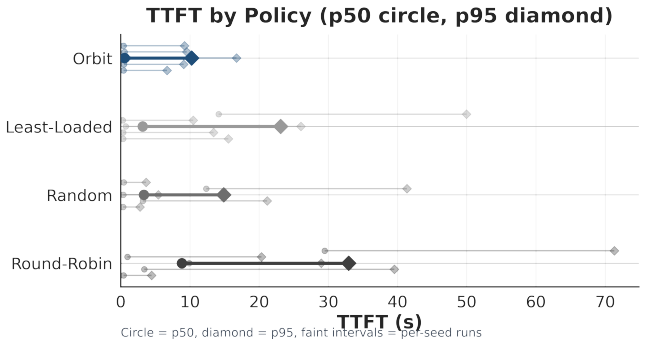

Figure 4: Median and p95 TTFT by policy. The gap between Orbit and the baselines is largest at the time-to-first-token boundary, where avoided prefill work matters most.

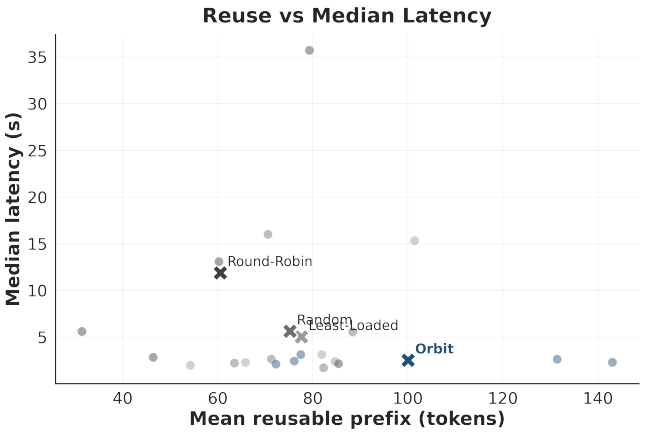

Figure 5: Reuse/latency tradeoff. Orbit occupies the desirable region: higher reuse and lower median latency.

5 Limitations and Impact

The results should be interpreted as evidence for a specific design point: approximate prefix summaries can improve online inter-cluster routing when routers have sparse reachability and incomplete cache state. They do not show that Orbit dominates all production routers. In particular, the latest benchmark does not directly deploy vLLM Production Stack prefix-aware routing, vLLM KV-cache-aware routing, or a global disaggregated KV-cache index. Those systems may have access to exact instance-level telemetry, LMCache state, or KV-transfer mechanisms that Orbit intentionally avoids. The impact on the project is that the central claim should re-

Table 3: Orbit traffic-class metrics from the latest run.

Traffic	Count/seed	TTFT p50	Lat. p50	Reuse
Agent	24.2	0.267	2.137	121.65
RAG	30.6	0.709	1.437	49.60
ShareGPT chat	41.2	0.915	3.344	117.99

main narrow: Orbit is a lightweight approximate-information router, not a replacement for engine-level or disaggregated KV-cache systems.

Deployment realism. The current prototype runs `llama.cpp` clusters on one local machine rather than on a multi-node GPU deployment with real inter-rack or inter-region links. The sparse topology and router-to-cluster costs emulate a multi-cluster environment, but they do not reproduce real network contention, GPU scheduling, Kubernetes pod churn, or heterogeneous accelerator behavior. This limits how strongly the project can claim production-scale latency improvements. A real deployment could change the relative importance of queue depth, RTT, prefill cost, and decode cost, so the measured coefficients should be treated as artifact-specific rather than portable production constants.

Workload representativeness. The mixed workload uses external ShareGPT-style chat, RAG, and agent/tool prompts, which is stronger than a synthetic-only benchmark. However, the final traffic mix is still a constructed workload. The amount of prefix overlap depends on how prompts are transformed, chunked, grouped by source, and replayed across seeds. If overlap is too high, even simple load-aware policies can receive accidental cache hits; if overlap is too low, no prefix-aware router can help much. This means the project should report overlap sensitivity before making a broad claim about realistic traffic. The manifest also records dataset paths and request counts, but not the traffic-mix weights, which weakens auditability.

Model and backend scale. The latest run uses a quantized 3B model through `llama.cpp`. Larger models, GPU-backed serving engines, and higher concurrency can change the prefill/decode ratio and batching behavior. Since Orbit's advantage comes largely from avoiding prefill work, the magnitude of the latency gain may increase or decrease under a different backend. The project can still claim that the routing mechanism works end-to-end with a live backend, but it should not claim that the exact latency deltas generalize to vLLM, TensorRT-LLM, SGLang, or production GPU clusters without further experiments.

Summary and cost-model limits. Orbit's summaries are approximate. Bloom filters can produce false positives, summaries can be stale, and hotsets only cover frequent prefixes. The router mitigates this with age penalties, uncertainty penalties, overlap thresholds, missing-summary penalties, and least-loaded fallback, but those mechanisms are guardrails rather than proofs of optimality. The cost model is also intentionally linear, so it cannot capture saturation, dynamic batching nonlinearities, or interactions between prompt length and queue depth. The latest run shows this limitation directly: calibration improved prediction MAE but was rejected because it worsened validation tail latency. The practical impact is that future auto-tuning must optimize routing outcomes, especially p95 TTFT and latency, rather than prediction error alone.

Baseline scope. The evaluated baselines are least-loaded, random, and round-robin inter-cluster routing. These are useful controls because they share the same topology, backend, and workload, but they are not the strongest possible production baselines. The project previously considered vLLM-inspired mocks, but they were removed from the primary evaluation because they were not faithful implementations of vLLM's routing boundary. This improves honesty but weakens the breadth of comparison. A stronger paper should benchmark against the actual vLLM Production Stack routers in their intended environment and then clearly separate instance-level results from inter-cluster results.

Robustness coverage. The implementation includes hooks for summary delay, gossip delay, summary drops, gossip drops, cluster outages, and retry penalties. The final reported benchmark disables those faults. As a result, the current evaluation supports the steady-state routing claim more strongly than the robustness claim. The project can argue that Orbit is designed to degrade toward least-loaded routing when summaries are missing or stale, but it still needs systematic failure sweeps to show how often that fallback is selected and how much performance degrades under control-plane disruption.

Measurement and reproducibility. The benchmark writes per-request traces, per-seed summaries, aggregate CSV/JSON files, calibration outputs, validation selections, and plots. This is enough to reproduce the reported aggregate numbers from the saved artifacts. Remaining gaps are the missing traffic-mix weights in the manifest, limited control-plane bandwidth validation, moderate measured request count, and the absence of a pinned external-dataset snapshot hash. These gaps do not invalidate the current results, but they reduce confidence that another researcher could exactly recreate the workload and control-plane overhead claims.

Overall impact. These limitations change the appropriate framing of the project. The strongest defensible claim is that approximate hierarchical prefix summaries are sufficient to outperform simple inter-cluster routing baselines on the latest mixed workload while improving both latency and reuse. The project should avoid stronger claims such as beating vLLM's

production router, proving bandwidth efficiency, or demonstrating datacenter-scale robustness until those experiments are added. The next most valuable work is therefore not another small tuning pass; it is a direct production-router comparison, an overlap-rate sweep, a summary-staleness sweep, a larger multi-node run, and fuller manifest recording.

6 Discussion

Why approximate summaries are useful. Exact global KV-cache state would be expensive to maintain because prompt caches change at request time scale. Orbitshows that exact state is not required to obtain useful routing decisions. Hierarchical Bloom filters provide a compact yes/no signal at multiple prefix depths, while hotsets reduce uncertainty for common short prefixes. The router does not need to know every cached block; it only needs enough information to distinguish likely high-reuse candidates from weak ones.

Staleness and safety. Approximate summaries create two risks: false positives and stale positives. Orbit's uncertainty term, summary-age penalty, missing-summary penalty, overlap threshold, and fallback margin all address those risks. The latest benchmark used no injected failures or delays, but the implementation supports summary delay, gossip delay, drop probabilities, and cluster outages. These robustness experiments are valuable future work for a paper-grade evaluation.

Calibration. The calibration result is nuanced. The fitted model dramatically improved prediction MAE on warm-up traces, but validation rejected it because p95 latency and TTFT worsened. This indicates that prediction accuracy alone is the wrong objective for a routing model. For LLM serving, tail-latency guardrails are necessary. Future tuning should optimize a routing-aware objective, such as validation p95 plus reuse, rather than only regression error.

7 Conclusion

Orbitdemonstrates that approximate, hierarchical prefix summaries can drive useful inter-cluster LLM routing decisions. In the latest live benchmark, Orbit outperforms least-loaded, random, and round-robin routing on both latency and prefix reuse. The result suggests a practical middle ground between prefix-blind load balancing and exact global KV-cache coordination. Rather than exporting complete KV state or migrating cache blocks, clusters can publish compact summaries, and routers can combine those summaries with load, staleness, uncertainty, and network cost.

The next step is a stronger production-style evaluation. The most important additions are direct comparison with vLLM Production Stack prefix/KV-aware routing, larger request counts, injected staleness and outage experiments, and a manifest that records every workload parameter. Even with those limitations, the current evidence supports the central research narrative: approximate prefix locality is enough to produce real end-to-end latency improvements in sparse multi-cluster LLM serving.

References

[1] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica. Efficient Memory Management for Large Language Model Serving with PagedAttention. SOSP 2023. vLLM design documentation.

[2] vLLM Production Stack Team. Prefix Aware Routing. vLLM Production Stack documentation.

[3] vLLM Production Stack Team. KV Cache Aware Routing. vLLM Production Stack documentation.

[4] vLLM Semantic Router Team. What is Semantic Router? vLLM Semantic Router documentation.

[5] vLLM Production Stack Team. Intelligent Semantic Routing. vLLM Production Stack documentation.

[6] Y. Mei, Y. Zhuang, X. Miao, J. Yang, Z. Jia, and R. Vinayak. Helix: Serving Large Language Models over Heterogeneous GPUs and Network via Max-Flow. ASPLOS 2025. paper page.

[7] Y. Zhong, S. Liu, J. Chen, J. Hu, Y. Zhu, X. Liu, X. Jin, and H. Zhang. DistServe: Disaggregating Prefill and Decoding for Goodput-optimized Large Language Model Serving. arXiv:2401.09670. paper page.

[8] P. Patel, E. Choukse, C. Zhang, A. Shah, I. Goiri, S. Maleki, and R. Bianchini. Splitwise: Efficient generative LLM inference using phase splitting. ISCA 2024. Microsoft Research page.

[9] R. Qin, Z. Li, W. He, M. Zhang, Y. Wu, W. Zheng, and X. Xu. Mooncake: A KVCache-centric Disaggregated Architecture for LLM Serving. arXiv:2407.00079. paper page.

[10] NVIDIA. KV Cache Transfer in Disaggregated Serving. NVIDIA Dynamo documentation.